7 - Diccionarios

Alejandro Veloz

aavelozb.github.io/py



Objetivos de la unidad

- **Asociar llaves con valores** en un diccionario: consultar, agregar, modificar y eliminar.
- **Elegir** entre una lista y un diccionario según el problema.
- Reemplazar una cascada de `if-elif` por **una consulta** al diccionario.
- **Contar y clasificar** datos por categoría.
- Combinar **diccionarios, listas y listas de listas**.

El problema

Una agenda guardada en dos listas paralelas:

```
nombres   = ['Ana', 'Luis', 'Paula']
telefonos = [5551428, 5550012, 5552437]
```

El problema

Una agenda guardada en dos listas paralelas:

```
nombres   = ['Ana', 'Luis', 'Paula']
telefonos = [5551428, 5550012, 5552437]
```

Para obtener el teléfono de Luis hay que **buscar su posición** primero.

```
i = nombres.index('Luis')
print(telefonos[i])      # 5550012
```

El problema

Una agenda guardada en dos listas paralelas:

```
nombres   = ['Ana', 'Luis', 'Paula']
telefonos = [5551428, 5550012, 5552437]
```

Para obtener el teléfono de Luis hay que **buscar su posición** primero.

```
i = nombres.index('Luis')
print(telefonos[i])      # 5550012
```

La posición no dice nada sobre el dato. Lo que interesa es el **nombre**.

El problema

Contar los votos de cada candidato con listas paralelas:

```
votos = ['Rojas', 'Soto', 'Rojas', 'Paz', 'Soto', 'Rojas']
candidatos = []
conteo = []
for v in votos:
    if v in candidatos:
        i = candidatos.index(v)
        conteo[i] += 1
    else:
        candidatos.append(v)
        conteo.append(1)
```

El problema

Contar los votos de cada candidato con listas paralelas:

```
votos = ['Rojas', 'Soto', 'Rojas', 'Paz', 'Soto', 'Rojas']
candidatos = []
conteo = []
for v in votos:
    if v in candidatos:
        i = candidatos.index(v)
        conteo[i] += 1
    else:
        candidatos.append(v)
        conteo.append(1)
```

Necesitamos llegar a un valor **por su nombre**, no por su posición.

Lo básico

Definición de diccionario

Un diccionario **asocia llaves con valores**. Se escribe entre **llaves** `{ }`, con pares `llave: valor` separados por comas.

```
patas = {'gato': 4, 'pulpo': 8, 'araña': 8, 'gallina': 2}
telefonos = {'Ana': 5551428, 'Luis': 5550012}
vacio = {}
```

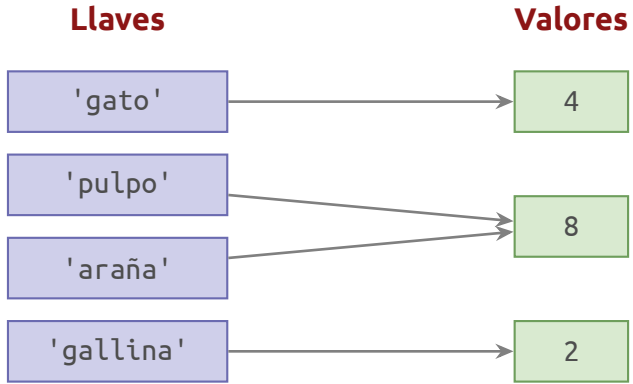
Definición de diccionario

Un diccionario **asocia llaves con valores**. Se escribe entre **llaves** `{ }`, con pares `llave: valor` separados por comas.

```
patas = {'gato': 4, 'pulpo': 8, 'araña': 8, 'gallina': 2}
telefonos = {'Ana': 5551428, 'Luis': 5550012}
vacio = {}
```

- Cada llave tiene **un solo** valor asociado.
- El diccionario vacío se escribe `{}` y tiene largo `0`.

Llaves y valores



- Dos llaves distintas pueden tener el **mismo valor**.
- Una llave **no se repite** y apunta a un solo valor.

Acceso por llave

`d[k]` entrega el valor asociado a la llave `k`.

```
patas = {'gato': 4, 'pulpo': 8, 'araña': 8, 'gallina': 2}
print(patas['pulpo'])     # 8
print(patas['perro'])     # KeyError: 'perro'
```

Acceso por llave

`d[k]` entrega el valor asociado a la llave `k`.

```
patas = {'gato': 4, 'pulpo': 8, 'araña': 8, 'gallina': 2}
print(patas['pulpo'])     # 8
print(patas['perro'])     # KeyError: 'perro'
```

- Es la notación de las listas, pero entre corchetes va la **llave**.
- Una llave que no existe produce `KeyError`.

Consulta previa con `in`

```
animal = input('Animal: ')
if animal in patas:
    print('Tiene', patas[animal], 'patas')
else:
    print('No conozco ese animal')
```

Consulta previa con `in`

```
animal = input('Animal: ')
if animal in patas:
    print('Tiene', patas[animal], 'patas')
else:
    print('No conozco ese animal')
```

`in` busca entre las llaves, no entre los valores.

```
print('gato' in patas)    # True
print(4 in patas)         # False
```

Agregar y modificar

La asignación `d[k] = v` sirve para las dos cosas.

```
notas = {'Ana': 55, 'Luis': 70}
notas['Paula'] = 48     # 'Paula' no existe: se agrega
notas['Luis'] = 75      # 'Luis' existe: se reemplaza
print(notas)            # {'Ana': 55, 'Luis': 75, 'Paula': 48}
```

Agregar y modificar

La asignación `d[k] = v` sirve para las dos cosas.

```
notas = {'Ana': 55, 'Luis': 70}
notas['Paula'] = 48     # 'Paula' no existe: se agrega
notas['Luis'] = 75      # 'Luis' existe: se reemplaza
print(notas)            # {'Ana': 55, 'Luis': 75, 'Paula': 48}
```

- Si la llave **no existe**, se agrega el par.
- Si la llave **ya existe**, cambia su valor y el valor antiguo se pierde.

Eliminar con `del`

`del d[k]` elimina la llave `k` junto con su valor.

```
notas = {'Ana': 55, 'Luis': 75, 'Paula': 48}
del notas['Luis']
print(notas)          # {'Ana': 55, 'Paula': 48}
del notas['Pedro']    # KeyError: 'Pedro'
```

Eliminar con `del`

`del d[k]` elimina la llave `k` junto con su valor.

```
notas = {'Ana': 55, 'Luis': 75, 'Paula': 48}
del notas['Luis']
print(notas)          # {'Ana': 55, 'Paula': 48}
del notas['Pedro']    # KeyError: 'Pedro'
```

- `del` recibe una **llave**, nunca una posición.
- Igual que con `d[k]`, conviene preguntar antes con `in`.

Operaciones sobre diccionarios

Operación	Resultado
`len(d)`	Cantidad de pares llave-valor
`k in d`	`True` si `k` es una llave de `d`
`k not in d`	`True` si `k` no es una llave de `d`
`d[k]`	Valor asociado a `k`
`d[k] = v`	Agrega el par o cambia el valor
`del d[k]`	Elimina el par

Operaciones sobre diccionarios

Operación	Resultado
`len(d)`	Cantidad de pares llave-valor
`k in d`	`True` si `k` es una llave de `d`
`k not in d`	`True` si `k` no es una llave de `d`
`d[k]`	Valor asociado a `k`
`d[k] = v`	Agrega el par o cambia el valor
`del d[k]`	Elimina el par

Un diccionario no se concatena con `+`, no se rebana y no tiene `sort`.

Tipos de llaves y valores

- Las llaves suelen ser **strings** o **enteros**.
- Los valores pueden ser **de cualquier tipo**, también listas.

```
camisetas = {1: 'Bravo', 7: 'Sánchez', 8: 'Vidal'}
cursos = {'Progra': ['Ana', 'Luis'], 'Mate': ['Paula']}
```

Tipos de llaves y valores

- Las llaves suelen ser **strings** o **enteros**.
- Los valores pueden ser **de cualquier tipo**, también listas.

```
camisetas = {1: 'Bravo', 7: 'Sánchez', 8: 'Vidal'}
cursos = {'Progra': ['Ana', 'Luis'], 'Mate': ['Paula']}
```

Una lista no puede ser llave, porque es mutable.

```
d = {}
d[[1, 2]] = 'hola'    # TypeError
```

Orden de un diccionario

- Un diccionario **no tiene posiciones**. `d[0]` busca la **llave** `0`.
- El `for` entrega las llaves en el **orden en que se agregaron**.
- Dos diccionarios con los mismos pares son iguales, en cualquier orden.

```
a = {'x': 1, 'y': 2}
b = {'y': 2, 'x': 1}
print(a == b)     # True
```

Orden de un diccionario

- Un diccionario **no tiene posiciones**. `d[0]` busca la **llave** `0`.
- El `for` entrega las llaves en el **orden en que se agregaron**.
- Dos diccionarios con los mismos pares son iguales, en cualquier orden.

```
a = {'x': 1, 'y': 2}
b = {'y': 2, 'x': 1}
print(a == b)     # True
```

Cuando el orden es parte del problema, se usa una lista.

Listas y diccionarios

	Lista	Diccionario
Se escribe	`[55, 70]`	`{'Ana': 55, 'Luis': 70}`
Se accede por	Posición `0, 1, 2, ...`	Llave
`in` busca entre	Los elementos	Las llaves
Agregar	`l.append(x)`	`d[k] = v`
Eliminar	`del l[i]`	`del d[k]`
¿Se puede modificar?	Sí	Sí

Listas y diccionarios

	Lista	Diccionario
Se escribe	`[55, 70]`	`{'Ana': 55, 'Luis': 70}`
Se accede por	Posición `0, 1, 2, ...`	Llave
`in` busca entre	Los elementos	Las llaves
Agregar	`l.append(x)`	`d[k] = v`
Eliminar	`del l[i]`	`del d[k]`
¿Se puede modificar?	Sí	Sí

Una lista guarda una **secuencia**. Un diccionario guarda una **asociación**.

Recorrer un diccionario

Recorrido con for

for recorre las **llaves**. El valor se obtiene con la llave.

```
capitales = {'Chile': 'Santiago', 'Perú': 'Lima', 'Ecuador': 'Quito'}
for pais in capitales:
    print('La capital de', pais, 'es', capitales[pais])
```

Recorrido con for

for recorre las **llaves**. El valor se obtiene con la llave.

```
capitales = {'Chile': 'Santiago', 'Perú': 'Lima', 'Ecuador': 'Quito'}
for pais in capitales:
    print('La capital de', pais, 'es', capitales[pais])
```

```
La capital de Chile es Santiago
La capital de Perú es Lima
La capital de Ecuador es Quito
```

Recorrido con for

for recorre las **llaves**. El valor se obtiene con la llave.

```
capitales = {'Chile': 'Santiago', 'Perú': 'Lima', 'Ecuador': 'Quito'}
for pais in capitales:
    print('La capital de', pais, 'es', capitales[pais])
```

```
La capital de Chile es Santiago
La capital de Perú es Lima
La capital de Ecuador es Quito
```

La variable del for toma cada **llave**, nunca el valor.

Acumular sobre los valores

El total de una boleta:

```
precios = {'pan': 1800, 'leche': 1100, 'huevos': 3200}
total = 0
for producto in precios:
    total += precios[producto]
print(total)      # 6100
```

Acumular sobre los valores

El total de una boleta:

```
precios = {'pan': 1800, 'leche': 1100, 'huevos': 3200}
total = 0
for producto in precios:
    total += precios[producto]
print(total)      # 6100
```

`sum(precios)` produce un error, porque intenta sumar las llaves, y las llaves son strings.

Llave del mayor valor

El producto más caro:

```
caro = ''
mayor = 0
for producto in precios:
    if precios[producto] > mayor:
        mayor = precios[producto]
        caro = producto
print(caro, mayor)      # huevos 3200
```

Llave del mayor valor

El producto más caro:

```
caro = ''
mayor = 0
for producto in precios:
    if precios[producto] > mayor:
        mayor = precios[producto]
        caro = producto
print(caro, mayor)      # huevos 3200
```

Se guarda la llave del mejor. Con ella se recupera cualquier dato asociado.

Recuperar y buscar

Recuperar el valor de una llave es una sola consulta.

```
camisetas = {1: 'Bravo', 7: 'Sánchez', 8: 'Vidal'}
print(camisetas[8])     # Vidal
```

Recuperar y buscar

Recuperar el valor de una llave es una sola consulta.

```
camisetas = {1: 'Bravo', 7: 'Sánchez', 8: 'Vidal'}
print(camisetas[8])     # Vidal
```

Buscar la llave de un valor obliga a recorrer el diccionario.

```
numero = -1
for n in camisetas:
    if camisetas[n] == 'Sánchez':
        numero = n
print(numero)           # 7
```

Recuperar y buscar

Recuperar el valor de una llave es una sola consulta.

```
camisetas = {1: 'Bravo', 7: 'Sánchez', 8: 'Vidal'}
print(camisetas[8])     # Vidal
```

Buscar la llave de un valor obliga a recorrer el diccionario.

```
numero = -1
for n in camisetas:
    if camisetas[n] == 'Sánchez':
        numero = n
print(numero)           # 7
```

Si hay que buscar por valor muchas veces, conviene invertir el diccionario.

Invertir un diccionario

Un diccionario nuevo, con los valores como llaves:

```
por_nombre = {}
for n in camisetas:
    por_nombre[camisetas[n]] = n
print(por_nombre)
# {'Bravo': 1, 'Sánchez': 7, 'Vidal': 8}
print(por_nombre['Sánchez'])     # 7
```

Invertir un diccionario

Un diccionario nuevo, con los valores como llaves:

```
por_nombre = {}
for n in camisetas:
    por_nombre[camisetas[n]] = n
print(por_nombre)
# {'Bravo': 1, 'Sánchez': 7, 'Vidal': 8}
print(por_nombre['Sánchez'])     # 7
```

Si dos llaves tienen el mismo valor, la inversión conserva sólo una. En `patas`, `'pulpo'` y `'araña'` tienen 8 patas.

Cambios durante el recorrido

No se agregan ni se eliminan llaves mientras se recorre.

```
notas = {'Ana': 55, 'Luis': 30, 'Paula': 48}
for nombre in notas:
    if notas[nombre] < 40:
        del notas[nombre]
# RuntimeError: dictionary changed size during iteration
```

Cambios durante el recorrido

No se agregan ni se eliminan llaves mientras se recorre.

```
notas = {'Ana': 55, 'Luis': 30, 'Paula': 48}
for nombre in notas:
    if notas[nombre] < 40:
        del notas[nombre]
# RuntimeError: dictionary changed size during iteration
```

Cambiar **valores** sí está permitido.

```
for nombre in notas:
    notas[nombre] += 5
```

Cambios durante el recorrido

No se agregan ni se eliminan llaves mientras se recorre.

```
notas = {'Ana': 55, 'Luis': 30, 'Paula': 48}
for nombre in notas:
    if notas[nombre] < 40:
        del notas[nombre]
# RuntimeError: dictionary changed size during iteration
```

Cambiar **valores** sí está permitido.

```
for nombre in notas:
    notas[nombre] += 5
```

Para eliminar, se construye un diccionario nuevo con los que se quedan.

En lugar de `if-elif`

Una cascada de `if-elif`

Nombre del mes a partir de su número:

```
mes = int(input('Mes: '))
if mes == 1:
    nombre = 'enero'
elif mes == 2:
    nombre = 'febrero'
elif mes == 3:
    nombre = 'marzo'
# ... nueve casos más
```

Una cascada de `if-elif`

Nombre del mes a partir de su número:

```
mes = int(input('Mes: '))
if mes == 1:
    nombre = 'enero'
elif mes == 2:
    nombre = 'febrero'
elif mes == 3:
    nombre = 'marzo'
# ... nueve casos más
```

Doce casos y veinticuatro líneas, todas casi iguales.

Una consulta al diccionario

```
meses = {1: 'enero', 2: 'febrero', 3: 'marzo', 4: 'abril',
         5: 'mayo', 6: 'junio', 7: 'julio', 8: 'agosto',
         9: 'septiembre', 10: 'octubre', 11: 'noviembre',
         12: 'diciembre'}

mes = int(input('Mes: '))
if mes in meses:
    print(meses[mes])
else:
    print('Mes inválido')
```

Una consulta al diccionario

```
meses = {1: 'enero', 2: 'febrero', 3: 'marzo', 4: 'abril',
         5: 'mayo', 6: 'junio', 7: 'julio', 8: 'agosto',
         9: 'septiembre', 10: 'octubre', 11: 'noviembre',
         12: 'diciembre'}

mes = int(input('Mes: '))
if mes in meses:
    print(meses[mes])
else:
    print('Mes inválido')
```

- Los **datos** quedan en el diccionario.
- La **lógica** se reduce a una consulta y una validación.

Reemplazo de palabras

Cambiar cada palabra que tenga sinónimo, y dejar las demás:

```
sinonimos = {'bonito': 'hermoso', 'cabello': 'pelo', 'miedo': 'temor'}
palabras = ['un', 'bonito', 'cabello', 'y', 'miedo']
nuevo = ''
for p in palabras:
    if p in sinonimos:
        nuevo += sinonimos[p] + ' '
    else:
        nuevo += p + ' '
print(nuevo)       # un hermoso pelo y temor
```

Reemplazo de palabras

Cambiar cada palabra que tenga sinónimo, y dejar las demás:

```
sinonimos = {'bonito': 'hermoso', 'cabello': 'pelo', 'miedo': 'temor'}
palabras = ['un', 'bonito', 'cabello', 'y', 'miedo']
nuevo = ''
for p in palabras:
    if p in sinonimos:
        nuevo += sinonimos[p] + ' '
    else:
        nuevo += p + ' '
print(nuevo)       # un hermoso pelo y temor
```

El diccionario funciona como una tabla de traducción.

Contar y clasificar

Conteo por categoría

Cada estudiante declara su mascota favorita.

```
favoritos = [['Ana', 'perro'], ['Luis', 'gato'], ['Paula', 'gato'],
             ['Pedro', 'pez'], ['Caro', 'gato'], ['Juan', 'perro']]
```

Queremos saber cuántos eligen cada mascota.

Conteo por categoría

Cada estudiante declara su mascota favorita.

```
favoritos = [['Ana', 'perro'], ['Luis', 'gato'], ['Paula', 'gato'],
             ['Pedro', 'pez'], ['Caro', 'gato'], ['Juan', 'perro']]
```

Queremos saber cuántos eligen cada mascota.

- Las **categorías** (perro, gato, pez) son las llaves.
- Los **contadores** son los valores.
- Una categoría nueva entra con su contador en `0`.

Conteo con diccionario

```
cuenta = {}
for nombre, animal in favoritos:
    if animal not in cuenta:
        cuenta[animal] = 0
    cuenta[animal] += 1
print(cuenta)    # {'perro': 2, 'gato': 3, 'pez': 1}
```

Conteo con diccionario

```
cuenta = {}
for nombre, animal in favoritos:
    if animal not in cuenta:
        cuenta[animal] = 0
    cuenta[animal] += 1
print(cuenta)    # {'perro': 2, 'gato': 3, 'pez': 1}
```

- Hay un contador por categoría.
- Las categorías **no se conocen de antemano**: aparecen al recorrer los datos.

Conteo paso a paso

Vuelta	`animal`	`cuenta['perro']`	`cuenta['gato']`	`cuenta['pez']`
1	`'perro'`	1		

Conteo paso a paso

Vuelta	animal	cuenta['perro']	cuenta['gato']	cuenta['pez']
1	'perro'	1		
2	'gato'	1	1	

Conteo paso a paso

Vuelta	animal	cuenta['perro']	cuenta['gato']	cuenta['pez']
1	'perro'	1		
2	'gato'	1	1	
3	'gato'	1	2	

Conteo paso a paso

Vuelta	animal	cuenta['perro']	cuenta['gato']	cuenta['pez']
1	'perro'	1		
2	'gato'	1	1	
3	'gato'	1	2	
4	'pez'	1	2	1

Conteo paso a paso

Vuelta	animal	cuenta['perro']	cuenta['gato']	cuenta['pez']
1	'perro'	1		
2	'gato'	1	1	
3	'gato'	1	2	
4	'pez'	1	2	1
5	'gato'	1	3	1

Conteo paso a paso

Vuelta	`animal`	`cuenta['perro']`	`cuenta['gato']`	`cuenta['pez']`
1	`'perro'`	1		
2	`'gato'`	1	1	
3	`'gato'`	1	2	
4	`'pez'`	1	2	1
5	`'gato'`	1	3	1
6	`'perro'`	2	3	1

Una casilla vacía indica que la llave todavía no existe. En las vueltas 1, 2 y 4 la llave se crea con valor `0` y de inmediato sube a `1`.

Llave sin inicializar

```
cuenta = {}
for nombre, animal in favoritos:
    cuenta[animal] += 1     # KeyError: 'perro'
```

Llave sin inicializar

```
cuenta = {}
for nombre, animal in favoritos:
    cuenta[animal] += 1     # KeyError: 'perro'
```

En la primera vuelta la llave `'perro'` no existe, y `+=` necesita leer su valor antes de sumarle uno.

Llave sin inicializar

```
cuenta = {}
for nombre, animal in favoritos:
    cuenta[animal] += 1     # KeyError: 'perro'
```

En la primera vuelta la llave `'perro'` no existe, y `+=` necesita leer su valor antes de sumarle uno.

Por eso va primero `if animal not in cuenta`.

Contar letras

```
def contar_letras(palabra):
    cuenta = {}
    for letra in palabra:
        if letra not in cuenta:
            cuenta[letra] = 0
        cuenta[letra] += 1
    return cuenta

print(contar_letras('banana'))   # {'b': 1, 'a': 3, 'n': 2}
```

Contar letras

```
def contar_letras(palabra):
    cuenta = {}
    for letra in palabra:
        if letra not in cuenta:
            cuenta[letra] = 0
        cuenta[letra] += 1
    return cuenta

print(contar_letras('banana'))   # {'b': 1, 'a': 3, 'n': 2}
```

La misma estructura cuenta en una lista, en un string o en las filas de una tabla.

Clasificación por categoría

Ahora queremos **los nombres** de quienes eligen cada mascota.

```
grupos = {}
for nombre, animal in favoritos:
    if animal not in grupos:
        grupos[animal] = []
    grupos[animal].append(nombre)
```

Clasificación por categoría

Ahora queremos **los nombres** de quienes eligen cada mascota.

```
grupos = {}
for nombre, animal in favoritos:
    if animal not in grupos:
        grupos[animal] = []
    grupos[animal].append(nombre)
```

```
{'perro': ['Ana', 'Juan'], 'gato': ['Luis', 'Paula', 'Caro'],
 'pez': ['Pedro']}
```

Clasificación por categoría

Ahora queremos **los nombres** de quienes eligen cada mascota.

```
grupos = {}
for nombre, animal in favoritos:
    if animal not in grupos:
        grupos[animal] = []
    grupos[animal].append(nombre)
```

```
{'perro': ['Ana', 'Juan'], 'gato': ['Luis', 'Paula', 'Caro'],
 'pez': ['Pedro']}
```

Cada valor es una lista que se construye con append.

Clasificación paso a paso

Vuelta	nombre	grupos['perro']	grupos['gato']	grupos['pez']
1	'Ana'	['Ana']		

Clasificación paso a paso

Vuelta	nombre	grupos['perro']	grupos['gato']	grupos['pez']
1	'Ana'	['Ana']		
2	'Luis'	['Ana']	['Luis']	

Clasificación paso a paso

Vuelta	nombre	grupos['perro']	grupos['gato']	grupos['pez']
1	'Ana'	['Ana']		
2	'Luis'	['Ana']	['Luis']	
3	'Paula'	['Ana']	['Luis', 'Paula']	

Clasificación paso a paso

Vuelta	nombre	grupos['perro']	grupos['gato']	grupos['pez']
1	'Ana'	['Ana']		
2	'Luis'	['Ana']	['Luis']	
3	'Paula'	['Ana']	['Luis', 'Paula']	
4	'Pedro'	['Ana']	['Luis', 'Paula']	['Pedro']

Clasificación paso a paso

Vuelta	nombre	grupos['perro']	grupos['gato']	grupos['pez']
1	'Ana'	['Ana']		
2	'Luis'	['Ana']	['Luis']	
3	'Paula'	['Ana']	['Luis', 'Paula']	
4	'Pedro'	['Ana']	['Luis', 'Paula']	['Pedro']
5	'Caro'	['Ana']	['Luis', 'Paula', 'Caro']	['Pedro']

Clasificación paso a paso

Vuelta	nombre	grupos['perro']	grupos['gato']	grupos['pez']
1	'Ana'	['Ana']		
2	'Luis'	['Ana']	['Luis']	
3	'Paula'	['Ana']	['Luis', 'Paula']	
4	'Pedro'	['Ana']	['Luis', 'Paula']	['Pedro']
5	'Caro'	['Ana']	['Luis', 'Paula', 'Caro']	['Pedro']
6	'Juan'	['Ana', 'Juan']	['Luis', 'Paula', 'Caro']	['Pedro']

Cuando la llave no existe, se crea con la lista vacía `[]` y después recibe el primer nombre.

Contar, acumular y clasificar

Tarea	Valor inicial	Actualización
Contar	`d[k] = 0`	`d[k] += 1`
Acumular	`d[k] = 0`	`d[k] += x`
Clasificar	`d[k] = []`	`d[k].append(x)`

Contar, acumular y clasificar

Tarea	Valor inicial	Actualización
Contar	`d[k] = 0`	`d[k] += 1`
Acumular	`d[k] = 0`	`d[k] += x`
Clasificar	`d[k] = []`	`d[k].append(x)`

Las tres comparten la misma estructura:

```
d = {}                    # antes del ciclo
for ... in datos:
    if k not in d:
        d[k] = ...        # valor inicial
    d[k] ...              # actualización
```

Acumular por categoría

Ventas del día por vendedor:

```
ventas = [['Rosa', 12000], ['Tomás', 8500], ['Rosa', 4300],
          ['Tomás', 2200], ['Inés', 9900]]
total = {}
for vendedor, monto in ventas:
    if vendedor not in total:
        total[vendedor] = 0
    total[vendedor] += monto
print(total)
# {'Rosa': 16300, 'Tomás': 10700, 'Inés': 9900}
```

Acumular por categoría

Ventas del día por vendedor:

```
ventas = [['Rosa', 12000], ['Tomás', 8500], ['Rosa', 4300],
          ['Tomás', 2200], ['Inés', 9900]]
total = {}
for vendedor, monto in ventas:
    if vendedor not in total:
        total[vendedor] = 0
    total[vendedor] += monto
print(total)
# {'Rosa': 16300, 'Tomás': 10700, 'Inés': 9900}
```

Es el conteo, pero se suma el monto en lugar de uno.

Ordenar el resultado

Un diccionario no se ordena. Para mostrar el resultado de mayor a menor se construye una **lista de listas** con el valor adelante:

```
ranking = []
for vendedor in total:
    ranking.append([total[vendedor], vendedor])
ranking.sort()
ranking.reverse()
print(ranking)
# [[16300, 'Rosa'], [10700, 'Tomás'], [9900, 'Inés']]
```

Ordenar el resultado

Un diccionario no se ordena. Para mostrar el resultado de mayor a menor se construye una **lista de listas** con el valor adelante:

```
ranking = []
for vendedor in total:
    ranking.append([total[vendedor], vendedor])
ranking.sort()
ranking.reverse()
print(ranking)
# [[16300, 'Rosa'], [10700, 'Tomás'], [9900, 'Inés']]
```

`sort` compara primero el monto, por eso el monto va en la primera posición.

Estructuras combinadas

Diccionario de listas

Cada producto tiene asociado su precio y su stock:

```
verduleria = {'Palta': [3800, 7], 'Tomates': [1200, 20],
              'Limones': [1500, 0], 'Papas': [950, 25]}
print(verduleria['Palta'])       # [3800, 7]
print(verduleria['Palta'][0])    # 3800   precio
print(verduleria['Palta'][1])    # 7      stock
```

Diccionario de listas

Cada producto tiene asociado su precio y su stock:

```
verduleria = {'Palta': [3800, 7], 'Tomates': [1200, 20],
              'Limones': [1500, 0], 'Papas': [950, 25]}
print(verduleria['Palta'])       # [3800, 7]
print(verduleria['Palta'][0])    # 3800   precio
print(verduleria['Palta'][1])    # 7      stock
```

`d[k][i]`: primero la llave elige la lista, después el índice elige el dato.

Compras en la verdulería

```
compras = ['Palta', 'Mangos', 'Limones', 'Papas']
total = 0
for p in compras:
    if p not in verduleria:
        print('No se vende', p)
    elif verduleria[p][1] == 0:
        print('No hay', p)
    else:
        total += verduleria[p][0]
        verduleria[p][1] -= 1
print('Total:', total)
```

Compras en la verdulería

```
compras = ['Palta', 'Mangos', 'Limones', 'Papas']
total = 0
for p in compras:
    if p not in verduleria:
        print('No se vende', p)
    elif verduleria[p][1] == 0:
        print('No hay', p)
    else:
        total += verduleria[p][0]
        verduleria[p][1] -= 1
print('Total:', total)
```

```
No se vende Mangos
No hay Limones
Total: 4750
```

Diccionario de diccionarios

```
numeros = {
    'Inglés':     {1: 'one',  2: 'two',  3: 'three'},
    'Portugués':  {1: 'um',   2: 'dois', 3: 'três'},
    'Mapudungún': {1: 'kiñe', 2: 'epu',  3: 'küla'}
}
print(numeros['Mapudungún'][3])     # küla
```

Diccionario de diccionarios

```
numeros = {
    'Inglés':     {1: 'one',  2: 'two',  3: 'three'},
    'Portugués':  {1: 'um',   2: 'dois', 3: 'três'},
    'Mapudungún': {1: 'kiñe', 2: 'epu',  3: 'küla'}
}
print(numeros['Mapudungún'][3])     # küla
```

Cómo se dice 2 en cada idioma:

```
for idioma in numeros:
    print(idioma, numeros[idioma][2])
```

Lista de diccionarios

```
alumnos = [
    {'nombre': 'Ana',   'carrera': 'Civil',     'nota': 62},
    {'nombre': 'Luis',  'carrera': 'Eléctrica', 'nota': 48},
    {'nombre': 'Paula', 'carrera': 'Civil',     'nota': 71}
]
for a in alumnos:
    if a['nota'] >= 55:
        print(a['nombre'], 'aprueba')
```

Lista de diccionarios

```
alumnos = [
    {'nombre': 'Ana',   'carrera': 'Civil',     'nota': 62},
    {'nombre': 'Luis',  'carrera': 'Eléctrica', 'nota': 48},
    {'nombre': 'Paula', 'carrera': 'Civil',     'nota': 71}
]
for a in alumnos:
    if a['nota'] >= 55:
        print(a['nombre'], 'aprueba')
```

- Cada diccionario es un **registro**.
- Las llaves dicen qué es cada dato, sin depender de su posición.

Alias y copia

Los diccionarios son mutables, igual que las listas.

```
a = {'x': 1, 'y': 2}
b = a            # alias: la misma estructura
b['x'] = 99
print(a)         # {'x': 99, 'y': 2}
```

Alias y copia

Los diccionarios son mutables, igual que las listas.

```
a = {'x': 1, 'y': 2}
b = a            # alias: la misma estructura
b['x'] = 99
print(a)         # {'x': 99, 'y': 2}
```

Una copia se construye recorriendo el original:

```
c = {}
for k in a:
    c[k] = a[k]
```

Ruteo

Ruteo de una función

```
def f(palabras):
    d = {}
    for p in palabras:
        if len(p) not in d:
            d[len(p)] = []
        d[len(p)].append(p)
    return d

print(f(['sol', 'mar', 'luna',
         'pez', 'nube']))
```

¿Qué imprime?

Ruteo de una función

```
def f(palabras):
    d = {}
    for p in palabras:
        if len(p) not in d:
            d[len(p)] = []
        d[len(p)].append(p)
    return d

print(f(['sol', 'mar', 'luna',
         'pez', 'nube']))
```

¿Qué imprime?

```
{3: ['sol', 'mar', 'pez'], 4:
['luna', 'nube']}
```

Clasifica las palabras por su largo.

Ruteo con alias

```
d = {'a': 1, 'b': 2}
e = d
d['c'] = 3
e['a'] = 10
del d['b']
print(len(e))
print(e)
print('b' in d)
```

¿Qué imprime?

Ruteo con alias

```
d = {'a': 1, 'b': 2}
e = d
d['c'] = 3
e['a'] = 10
del d['b']
print(len(e))
print(e)
print('b' in d)
```

¿Qué imprime?

```
2
{'a': 10, 'c': 3}
False
```

`e` es un alias de `d`: los cambios por cualquiera de los dos nombres afectan al mismo diccionario.

Errores frecuentes

Encuentre los tres errores

Mostrar los números que aparecen más de una vez, con su cantidad.

```
nros = [3, 1, 3, 2, 1, 3]
for n in nros:
    cuenta = {}
    if n not in cuenta:
        cuenta[n] = 0
    cuenta[n] += 1
for n in cuenta:
    if n > 1:
        print(n, 'aparece', cuenta(n), 'veces')
```

Encuentre los tres errores

(1) `cuenta = {}` está dentro del ciclo, y en cada vuelta se pierde lo contado. Va antes del `for`.

Encuentre los tres errores

(1) `cuenta = {}` está dentro del ciclo, y en cada vuelta se pierde lo contado. Va antes del `for`.

(2) `if n > 1` compara la **llave**. Debe ser `if cuenta[n] > 1`.

Encuentre los tres errores

(1) `cuenta = {}` está dentro del ciclo, y en cada vuelta se pierde lo contado. Va antes del `for`.

(2) `if n > 1` compara la **llave**. Debe ser `if cuenta[n] > 1`.

(3) `cuenta(n)` intenta llamar al diccionario como una función y produce `TypeError`. Debe ser `cuenta[n]`.

Encuentre los tres errores

(1) `cuenta = {}` está dentro del ciclo, y en cada vuelta se pierde lo contado. Va antes del `for`.

(2) `if n > 1` compara la **llave**. Debe ser `if cuenta[n] > 1`.

(3) `cuenta(n)` intenta llamar al diccionario como una función y produce `TypeError`. Debe ser `cuenta[n]`.

Corregido, imprime `3 aparece 3 veces` y `1 aparece 2 veces`.

Encuentre el error

Saber si Vidal está en la alineación.

```
camisetas = {1: 'Bravo', 7: 'Sánchez', 8: 'Vidal'}
if 'Vidal' in camisetas:
    print('Vidal juega')
else:
    print('Vidal no juega')     # se imprime esto
```

Encuentre el error

Saber si Vidal está en la alineación.

```
camisetas = {1: 'Bravo', 7: 'Sánchez', 8: 'Vidal'}
if 'Vidal' in camisetas:
    print('Vidal juega')
else:
    print('Vidal no juega')     # se imprime esto
```

`in` busca entre las llaves, que son números. Hay que comparar los valores.

```
juega = False
for n in camisetas:
    if camisetas[n] == 'Vidal':
        juega = True
```

Encuentre el error

Mostrar el primer precio de la lista.

```
precios = {'pan': 1800, 'leche': 1100, 'huevos': 3200}
print(precios[0])     # KeyError: 0
```

Encuentre el error

Mostrar el primer precio de la lista.

```
precios = {'pan': 1800, 'leche': 1100, 'huevos': 3200}
print(precios[0])     # KeyError: 0
```

Un diccionario no tiene posiciones: `precios[0]` busca la llave `0`, que no existe.

Encuentre el error

Mostrar el primer precio de la lista.

```
precios = {'pan': 1800, 'leche': 1100, 'huevos': 3200}
print(precios[0])     # KeyError: 0
```

Un diccionario no tiene posiciones: `precios[0]` busca la llave `0`, que no existe.

Se accede con la llave:

```
print(precios['pan'])    # 1800
```

Lo que usamos en esta unidad

SÍ

- Creación de diccionarios, vacíos o con pares.
- Acceso `d[k]`, asignación `d[k] = v` y `del d[k]`.
- `len`, `in` y `not in` sobre las llaves.
- Recorrido con `for` sobre las llaves.
- Contar, acumular y clasificar por categoría.
- Diccionarios de listas, de diccionarios y listas de diccionarios.

NO

- Métodos `keys()`, `values()` e `items()`.
- Métodos `get()` y `setdefault()`.
- Tuplas como llaves.
- Ordenar un diccionario.

Para ordenar un resultado, se construye una lista de listas desde el diccionario y se usa `sort()`.

Ejercicios

Ejercicios de calentamiento

(a) Dada una lista de palabras, construya un diccionario que asocie cada palabra con su largo.

(b) Dada una lista de enteros, construya un diccionario con dos llaves, `'par'` e `'impar'`, que cuente cuántos hay de cada tipo.

(c) Escriba una función que reciba un diccionario y un valor, y retorne la lista de las llaves que tienen ese valor.

(d) Escriba una función que reciba un string y retorne la letra que más se repite.

Ejercicio: cumpleaños por mes

```
datos = [['Andrea', '07-09-1994'], ['Federico', '29-09-1970'],
         ['Javiera', '02-07-2010'], ['Ignacio', '02-03-2007'],
         ['Benjamín', '14-09-2005'], ['Pedro', '23-03-1954']]
```

(a) `cumpleanos(datos)`: retorna un diccionario cuya llave es el mes (`'09'`) y cuyo valor es la lista de nombres que están de cumpleaños ese mes.

```
>>> cumpleanos(datos)
{'09': ['Andrea', 'Federico', 'Benjamín'], '07': ['Javiera'],
 '03': ['Ignacio', 'Pedro']}
```

(b) Repita (a), pero con el nombre del mes como llave (`'Septiembre'`) y una lista de `[día, nombre]` ordenada por día como valor. Use un diccionario `meses` para traducir el número.

Ejercicio: autopistas

Cada tránsito registra la patente de un vehículo y el número del pórtico por el que pasó.

```
transitos = [['BBJJ77', 2], ['CCHH19', 3], ['AAFF37', 3], ['BBJJ77', 1],
             ['AAFF37', 1], ['DDKK33', 3], ['AAFF37', 1], ['AAFF37', 2]]
```

(a) `mayor_movilidad(transitos)`: retorna una lista de listas con los tres vehículos que registran más pasadas por pórticos, de mayor a menor. Los empates se resuelven de cualquier forma.

```
>>> mayor_movilidad(transitos)
[['AAFF37', 4], ['BBJJ77', 2], ['DDKK33', 1]]
```

Ejercicio: autopistas

```
transitos = [['BBJJ77', 2], ['CCHH19', 3], ['AAFF37', 3], ['BBJJ77', 1],
             ['AAFF37', 1], ['DDKK33', 3], ['AAFF37', 1], ['AAFF37', 2]]
```

(b) `reportar(transitos)`: retorna un diccionario cuya llave es el pórtico y cuyo valor es la lista de patentes que pasaron por él, ordenada y sin repetidos.

```
>>> reportar(transitos)
{2: ['AAFF37', 'BBJJ77'], 3: ['AAFF37', 'CCHH19', 'DDKK33'],
 1: ['AAFF37', 'BBJJ77']}
```

Ejercicio: series de TV

```
# [nombre, pais, rating, géneros]
series = [['Game of Thrones', 'USA', 9.4, ['ficcion']],
          ['24', 'USA', 8.4, ['accion', 'suspenso']],
          ['La casa de papel', 'España', 9.2, ['accion', 'suspenso', 'drama']],
          ['Dark', 'Alemania', 9.2, ['ficcion', 'drama']],
          ['Sherlock', 'UK', 9.2, ['policial', 'drama', 'suspenso']],
          ['Heroes', 'USA', 7.7, ['ficcion', 'accion']],
          ['Misfits', 'UK', 8.4, ['accion', 'drama', 'ficcion']]]
```

(a) `series_por_genero(series)`: retorna un diccionario cuya llave es el género y cuyo valor es la lista de series de ese género, en orden alfabético.

(b) `paises_con_mas_series(series)`: retorna la lista de los dos países con más series, de mayor a menor. Use un diccionario para contar.

```
>>> paises_con_mas_series(series)
['USA', 'UK']
```

Ejercicio: Copa América

```
chile = {1: 'Bravo', 4: 'Isla', 17: 'Medel', 8: 'Vidal',
         20: 'Aránguiz', 7: 'Sánchez', 11: 'Vargas'}
argentina = {1: 'Romero', 4: 'Mercado', 14: 'Mascherano',
             10: 'Messi', 9: 'Higuaín', 7: 'Di María'}
```

(a) Muestre la alineación de un equipo con el formato `Con el número 8, Vidal`.

(b) Escriba una función que reciba un equipo y el nombre de un jugador, y retorne su número, o `-1` si no está.

(c) Muestre las parejas de jugadores que usan el mismo número en los dos equipos.

```
1 Bravo Romero
4 Isla Mercado
7 Sánchez Di María
```

Desafío: anagramas

Dos palabras son **anagramas** si tienen las mismas letras en otro orden, como `'roma'` y `'amor'`.

Escriba una función que reciba una lista de palabras y retorne un diccionario que agrupe las que son anagramas entre sí.

```
>>> anagramas(['roma', 'amor', 'sol', 'mora', 'los', 'luna'])
{'amor': ['roma', 'amor', 'mora'], 'los': ['sol', 'los'],
 'alnu': ['luna']}
```

Desafío: anagramas

Dos palabras son **anagramas** si tienen las mismas letras en otro orden, como `'roma'` y `'amor'`.

Escriba una función que reciba una lista de palabras y retorne un diccionario que agrupe las que son anagramas entre sí.

```
>>> anagramas(['roma', 'amor', 'sol', 'mora', 'los', 'luna'])
{'amor': ['roma', 'amor', 'mora'], 'los': ['sol', 'los'],
 'alnu': ['luna']}
```

Sugerencia: la llave de cada grupo son las letras de la palabra en orden alfabético. Se obtiene con `list(p)`, `sort()` y un acumulador de strings.